Parallelized Boid Simulation

Josiah Miggiani, Ryan Huang
15-418 Parallel Computer Architecture and Computing, Spring 2025
Carnegie Mellon University

Project Webpage

Summary

Boids, short for “bird-oids,” are independent actors which can collectively simulate the complex aggregate motion of a flock of birds, a school of fish, or even crowd movement of humans.

In our project, we explore three different approaches to parallelizing boids simulation: naive, octree, and spatial hash.

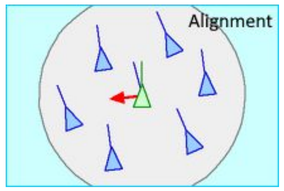
We leverage spatially-aware shared memory data structures to optimize finding neighbor boids, and ultimately reduce algorithmic complexity from $O(N^2)$ to $O(NlogN)$

Background

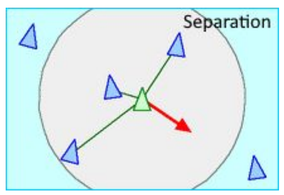
Boids were first proposed by Craig W. Reynolds in his paper submission to SIGGRAPH ‘87.

*“The aggregate motion of the simulated flock is the result of the **dense interaction** of the **relatively simple behaviors** of the individual simulated birds.”*

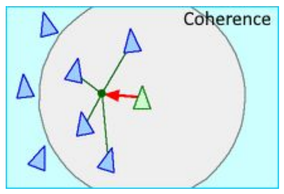
Though each boid is independently responsible for deciding the course of motion it takes, these decisions are influenced by each boid’s local perception of a highly dynamic environment.



Boid alignment seeks to maintain a similar velocity to nearby flockmates. Velocity is a vector quantity, representing heading (orientation) and speed.



Separation requires that each boid attempts to avoid flying into other boids. As a boid nears in spatial proximity to another, it will alter its heading to avoid collision.



Coherence dictates that boids seek to fly closer towards nearby boids. More specifically, it makes a boid want to be near the center of the flock.

Challenges

Workload

Communication: Boids constantly take lots of information from their surroundings. There is a high communication to computation ratio.

Divergent Execution: The information that a boid gathers leads to divergent decisions as boids decide whether to flock or avoid collision.

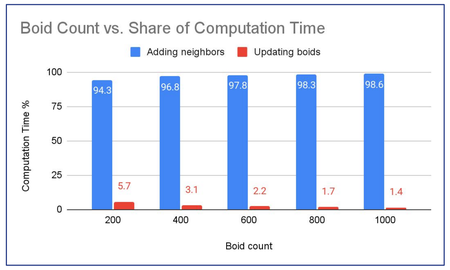
Constraints

Stale Data: Boids act upon shared data. This data will be dynamically updated as the simulation proceeds. We must ensure boids read from stale data.

Efficiency: We must ensure that accessing data is done efficiently. Data access patterns can be irregular, especially as shared data and boids move around.

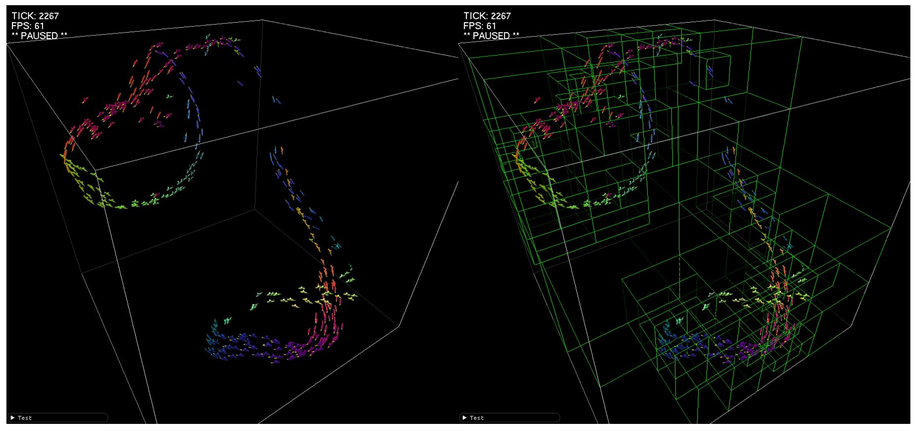
Approach

An initial investigation into the algorithm reveals that **finding neighbors dominates computation time**. This share only grows as boid counts increase. As such, we target spatial binning methods to aid in parallelizing adding neighbors.



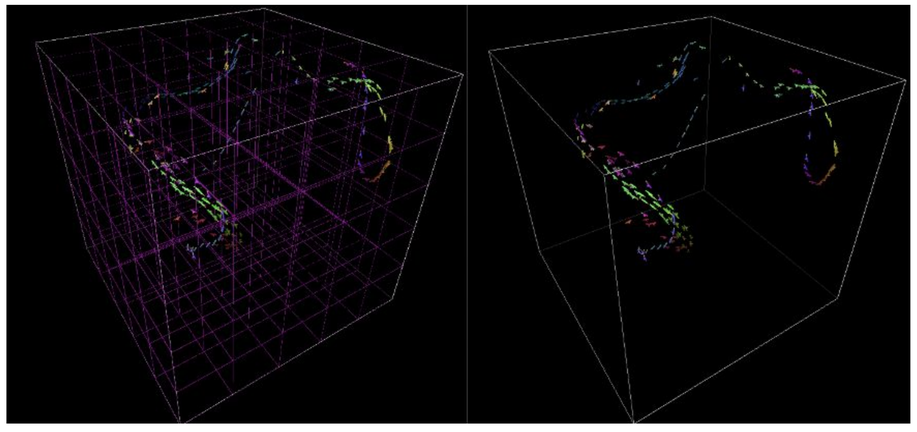
Octree

On insertion, boids traverses down from the root node through valid children octrees until there is capacity available at a leaf node.
When searching for a boid’s neighbors, the octree is traversed once more. At each level, all child octrees’ bounding boxes are tested to be within the boid’s neighborhood radius. If there is no box-sphere intersection, that octree and all its children are recursively ruled out, reducing the search space.
We parallelize neighbor queries, assigning each boid to a thread for a thread-safe, read-only octree search.



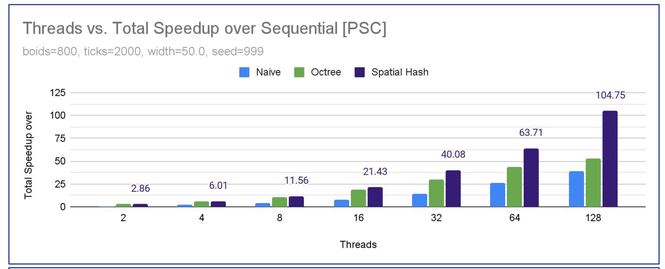
Hash

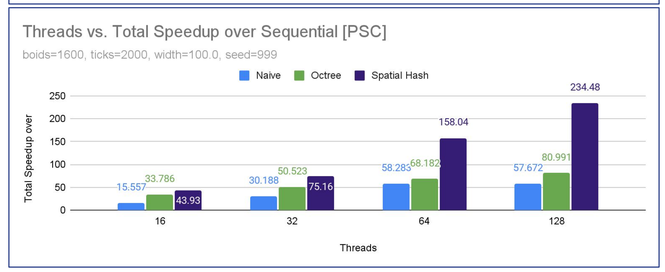
The hash partitions the simulation box into larger cells. The boids are then hashed and binned by cell location. Boids then query for a shortlist of candidate neighbors by accessing cells within vision radius. The hash is updated in parallel during simulation time, so fine grain locking is used to ensure thread safety.



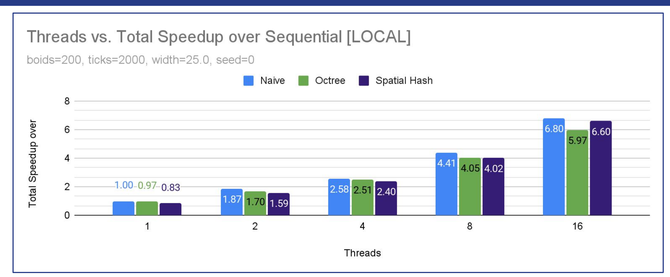
Results

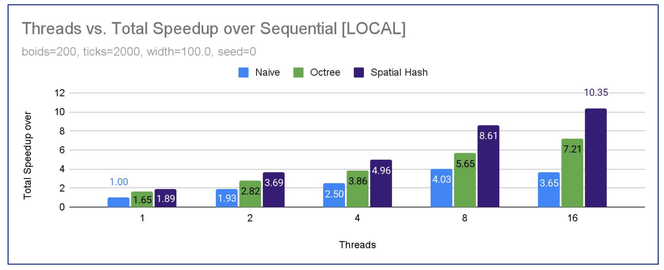
PSC: Bridges-2





LOCAL





Conclusions & Additional Information

- Octrees and hashing demonstrate remarkable performance improvements over the sequential/naive parallel implementations.
  - Hashing is very powerful, reaching a **maximum speedup of 234x.**
- This **advantage** is maximized for **increasingly complex simulations**, such as those with higher boid counts and wider simulation boundaries.
  - For simpler simulations, overhead costs cannot be effectively amortized, and naive parallelization can outperform.
- **Spatially-aware data structures** are well suited for parallelizing highly dynamic simulations.